

# Functions

# Function

- A function is similar to a program that consists of a group of statements that are intended to perform a specific task.
- The main purpose of a function is to perform a specific task or work.
- The function which is created by programmer is called user-define function.
- A function is a block of code which only runs when it is called.

# Function

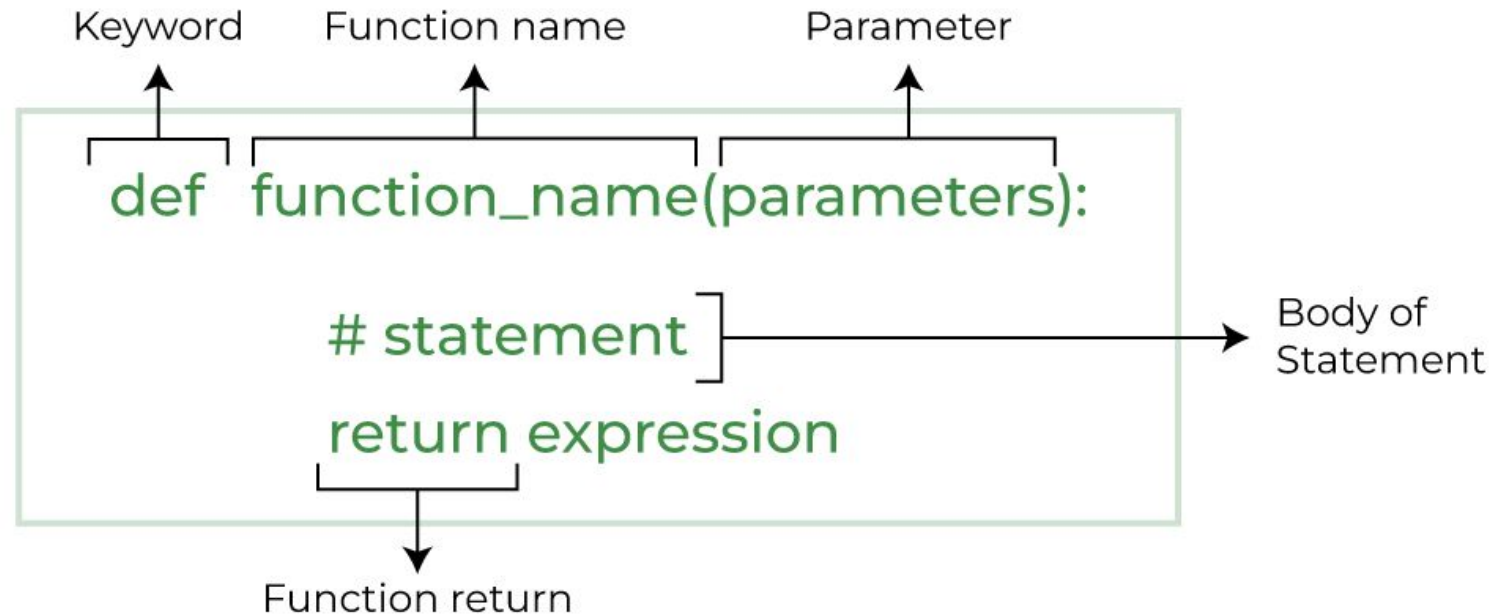
- Advantages of Function:
  1. Function are used to process data, make calculation or perform any task which is required in the software development.
  2. Once a function is written, it can be reused as and when required. So function are also called reusable code. Because of this reusability, the programmer can avoid code redundancy.
  3. Code maintenance will become easy because of function.
  4. Function provide modularity for programming. Module represent a part of programme. Usually, a programmer divides the main task into smaller sub tasks called modules. To represent each module, the programmer will develop a separate function.
  5. When there is an error in the software, the corresponding function can be modified without disturbing the other function in the software.
  6. The use of function in program will reduce the length of the program.

# Difference between a function and a Method

- A function can be written individually in a python program. A function is called using its name.
- When a function is written inside a class, it becomes a 'method'. A method is called using following ways:
  - Objectname.methodname()
  - Classname.methodname()
- So a function and method are same except their placement and the way they are called.
- Once a function is defined, it can be used by calling the function.

# Defining a Function

- We can define a function using 'def' keyword followed by function name.
- After the function name, we should write parentheses() which may contain parameters.
- Syntax of Function:



# Types of Functions in Python

- **Built-in library function:** These are Standard functions in Python that are available to use.
- **User-defined function:** We can create our own functions based on our requirements.
- User-defined function Example:

```
def my_function():  
    print("Hello from a function")  
  
my_function()
```

Output

Hello from a function

# Defining a Function

```
def sum(a,b):  
    c=a+b  
    print(c)
```

- In example, def represent the starting of function definition. 'sum' is the name of the function. Parantheses () denote that it is a function.
- Two variable a and b are called parameters.
- A parameter is a variable that receives data from outside into a function. So sum function can receives two values from outside and those values are stored into variable 'a' and 'b'.
- After parentheses (), colon (:) represents the beginning of the function body.
- Function body contains a group of statements, which constitute the logic of the function.

# Calling a function

- A function cannot run on its own. It runs only when we call it.
- While calling the function, we should pass the necessary value to the function in the parentheses as:

```
sum(10,15)
```

- Here, we are calling sum function and passing two values 10 and 15 to that function.
- When this statement is executed, the python interpreter jumps to the function definition and copies the value 10 and 15 into the parameter a and b respectively.

# Calling a function

- Example:

```
#Program to add to number using function  
def sum(a,b):  
    c= a+b  
    print("sum: ",c)  
  
sum(10,15) #Function calling  
sum(11.5,3.5)
```

```
sum: 25  
sum: 15.0
```

# Returning Result from a function

- We are return the result or output from the function using 'return' statement in the body of the function.
- Example: return c (Return c value out of function)  
          return 100 (Return 100 value)  
          return x, y (Return 2 values)
- When a function does not return any result, we need not write the return statement in the body of the function.

# Returning Result from a function

- Exam

```
# Program to find sum of two number and return result  
def sum(a,b):  
    c=a+b  
    return c  
x=sum(10,15)  
print(x)  
y=sum(6.7,11)  
print(y)
```

25

17.7

# Example:

```
#Program to calculate factorial
```

```
def factorial(n):  
    fact=1  
  
    while n>=1:  
        fact=fact*n  
        n-=1  
    print(fact)
```

```
factorial(int(input("Enter number")))
```

Enter number4

24

```
#A function to test weather a number is odd or even
```

```
def odd_even(n):  
    if(n%2==0):  
        print(n,"is even")  
    else:  
        print(n,"is odd")
```

```
odd_even(11)    #function calling  
odd_even(int(input("enter number")))
```

11 is odd

enter number12

12 is even

# Formal and Actual Arguments

- When a function is defined, it may have some parameters. These parameters are useful to receive values from outside of the function. They are called 'formal arguments'.
- When we call the function, we should pass data or values to the function. These values are called 'actual arguments'.

```
def sum(a,b): # a, b are formal parameter
    c=a+b
    print(c)

x=10; y=15
sum(x,y) # x, y are actual parameter
```

# Formal and Actual Arguments

- The actual arguments are used in a function call are of 4 types.
  1. Positional arguments
  2. Keyword arguments
  3. Default arguments
  4. Variable length arguments

# Types of Python Function Arguments

## 1. Positional Arguments

- These are the arguments passed to a function in correct positional order.
- Here, the number of arguments and their positions in the function definition should match exactly with the number and position of the argument in the function call.

## 2. Keyword Arguments

- Keyword arguments are arguments that identify the parameters by their names.
- Example: `def sum(a,b):`
- We can write the definition of function sum with two arguments 'a' and 'b'.
- At the time of calling this function, we have to pass two values and we can mention which value is for what.
- `Sum(a=10,b=15)`

# Types of Python Function Arguments

## 3. Default Arguments

- We can mention some default value for the function parameters in the definition of sum() function.  
def sum(a=10,b)
- Here, the first argument is 'a' whose default value is mentioned to be 10, but the second argument 'b' whose default value is not mentioned.

```
def myFun(x, y=50):  
    print("x: ", x)  
    print("y: ", y)
```

```
myFun(10)
```

Output

```
x:  10  
y:  50
```

## 4. Variable Length Arguments / Arbitrary arguments

- In Python Arbitrary Keyword Arguments, `*args`, and `**kwargs` can pass a variable number of arguments to a function using special symbols. There are two special symbols:
- **`*args`** in Python (Non-Keyword Arguments)
- **`**kwargs`** in Python (Keyword Arguments)

```
def myFun(*argv):  
    for arg in argv:  
        print(arg, end=" ")
```

```
myFun('Hello', 'Welcome', 'to', 'python class')
```

### Output

```
Hello Welcome to python class  
=== Code Execution Successful ===
```

```
def myFun(**kwargs):  
    for key, value in kwargs.items():  
        print("%s == %s" % (key, value))
```

```
myFun(first='Welcome', mid='to', last='Python class')
```

### Output

```
mid == Welcome  
first == to  
last == Python class
```

# Docstring

- The first string after the function is called the Document string or Docstring in short.
- This is used to describe the functionality of the function.
- The use of docstring in functions is optional but it is considered a good practice.
- The below syntax can be used to print out the docstring of a function.
- Syntax: `print(function_name.__doc__)`

- Example:

```
def evenOdd(x):  
    """Function to check if the number is even or  
    odd"""  
    if (x % 2 == 0):  
        print("even")  
    else:  
        print("odd")  
  
print(evenOdd.__doc__)
```

### Output

Function to check if the number is even or odd

# Function examples

- Write python function to print sum and average of **n** numbers entered by user
- Write python function to find maximum of three numbers.
- Write python function to display Fibonacci series.
- Write python function to display sum of string separated by “10#20#30#40”
- Write python function to display Sum of Digits

# Write python program to print sum and average of n numbers

```
def sum_and_average(n):  
    total = 0  
    for i in range(n):  
        num = int(input("Enter number"))  
        total += num  
  
    avg = total / n  
  
    print("Sum =", total)  
    print("Average =", avg)  
  
sum_and_average(5)
```

## Output

```
Enter number5  
Enter number5  
Enter number1  
Enter number2  
Enter number3  
Sum = 16  
Average = 3.2
```

Write python function to find maximum of three numbers.

```
def maximum_of_three(a, b, c):  
    if a >= b and a >= c:  
        return a  
    elif b >= a and b >= c:  
        return b  
    else:  
        return c
```

```
print("Maximum is:", maximum_of_three(10, 25, 15))
```

Output

Maximum is: 25

Write python function to display Fibonacci series

```
def fibonacci(n):  
    a = 0  
    b = 1  
    # Check if n is less than 0  
    if n < 0:  
        print("Incorrect input")  
  
    # Check if n is equal to 0  
    elif n == 0:  
        return 0  
  
    # Check if n is equal to 1  
    elif n == 1:  
        return b  
    else:  
        for i in range(1, n+1):  
            print(a)  
            c = a + b  
            a = b  
            b = c  
  
fibonacci(9)
```

| Output |
|--------|
| 0      |
| 1      |
| 1      |
| 2      |
| 3      |
| 5      |
| 8      |
| 13     |
| 21     |

Write python function to display sum of string separated by "10#20#30#40"

```
def sum_of_string(s):  
    parts = s.split("#")  
    total = 0  
    for num in parts:  
        total += int(num)  
    return total  
  
s = "10#20#30#40"  
print("Sum =", sum_of_string(s))
```

Output

Sum = 100

# Write python function to display Sum of Digits

```
def sum_of_digits(n):  
    total = 0  
    while n > 0:  
        digit = n % 10  
        total += digit  
        n //= 10  
    return total
```

```
print("Sum of digits of 1234 =", sum_of_digits(1234))
```

## Output

Sum of digits of 1234 = 10

# Python function to reverse a number.

```
def reverse_number(n):  
    rev = 0  
    while n > 0:  
        digit = n % 10  
        rev = rev * 10 + digit  
        n = n // 10  
    return rev  
  
print(reverse_number(12345))  # 54321
```

Output

54321

# Python Function within Functions

- A function that is defined inside another function is known as the **inner function** or **nested function**.
- Nested functions can access variables of the enclosing scope.

```
def outer_function():  
    print("This is the outer function")  
  
    def inner_function():  
        print("This is the inner function")  
  
    inner_function()  
  
outer_function()
```

## Output

```
This is the outer function  
This is the inner function
```

# Python Function within Functions

```
def f1():  
    s = 'Hello World'  
  
    def f2():  
        print(s)  
    f2()  
  
f1()
```

Hello World

=== Code Execution Successful ===

# Returning Inner Function

```
def outer_function():  
  
    def inner_function():  
        return "Hello from inner function!"  
    return inner_function  
  
my_func = outer_function()  
print(my_func())
```

## Output

Hello from inner function!

# Passing Values to Inner Function

```
def calculator(a, b):  
    def add():  
        return a + b  
  
    def multiply():  
        return a * b  
  
    print("Addition:", add())  
    print("Multiplication:", multiply())  
  
calculator(5, 3)
```

## Output

Addition: 8

Multiplication: 15

# Global vs Local Variables

## **Local Variable :**

- When we declare a variable inside a function, it became a local variable. A local variable is a variable whose scope is limited only to that function it is created.
- That means the local variable value is available only in that function and not outside of that function.

## **Global Variable :**

- When a variable declare above a function, it becomes global variable. Such variables are available to all the functions which are written after it.
- The scope of the global variable is for the entire programme.

# Global vs Local Variables

- **Global Variable:** Declared outside all functions. Accessible anywhere in the code (unless shadowed by local variable of same name).
- **Local Variable:** Declared inside a function. Accessible only inside that function.
- Example 1: Local Variable

```
def my_function():  
    x = 10    # local variable  
    print("Inside function:", x)
```

```
my_function()
```

```
print(x)    # ✗ Error: x is local, not accessible  
            outside
```

## Output

ERROR!

Inside function: 10

Traceback (most recent call last):

File "<main.py>", line 6, in <module>

NameError: name 'x' is not defined

# Global vs Local Variables

- Example 2: Global Variable

```
x = 50 # global variable

def my_function():
    print("Inside function:", x)

my_function()
print("Outside function:", x)
```

## Output

```
Inside function: 50
Outside function: 50
```

# Global vs Local Variables

- Example 3: Local Variable Overrides Global

```
x = 100 # global variable

def my_function():
    x = 200 # local variable (different from
            global)
    print("Inside function:", x)

my_function()
print("Outside function:", x)
```

## Output

```
Inside function: 200
Outside function: 100
```

# Global vs Local Variables

- Example 4: Using global Keyword

```
x = 5 # global variable

def my_function():
    global x
    x = 20 # modifies global variable
    print("Inside function:", x)

my_function()
print("Outside function:", x)
```

## Output

```
Inside function: 20
Outside function: 20
```

# Global vs Local Variables

- Example 5: Local + Global Together

```
x = 10

def my_function():
    y = 5    # local variable
    print("Global x inside function:", x)
    print("Local y inside function:", y)

my_function()
print("Global x outside function:", x)
# print(y)  # ✗ Error: y is local
```

## Output

```
Global x inside function: 10
Local y inside function: 5
Global x outside function: 10
```

# Returning Multiple values from a Function

- A function returns a single value in the programming language like C or Java. But in python, a function can return multiple values. For that we have to use return statement as:

Return a,b,c

- In above statement, three values which are in a, b and c are returned. These values are returned by the function as a tuple.

```
#Returning multiple value to function
def operation(a,b):
    x=a+b
    y=a-b
    z=a*b
    return x,y,z

A,B,C=operation(2,3)
print("Addition: ",A)
print("Suntraction: ",B)
print("Multiplication: ",C)
```

```
Addition:  5
Suntraction:  -1
Multiplication:  6
```

# Returning Multiple Values as a Tuple

```
def calculate(a, b):  
    add = a + b  
    sub = a - b  
    mul = a * b  
    return add, sub, mul  
  
result = calculate(10, 5)  
print(result)  
print(result[0])
```

## Output

```
(15, 5, 50)  
15
```

# Unpacking Returned Values

```
def calculate(a, b):  
    return a + b, a - b, a * b  
  
x, y, z = calculate(8, 3)  
print("Addition:", x)  
print("Subtraction:", y)  
print("Multiplication:", z)
```

## Output

```
Addition: 11  
Subtraction: 5  
Multiplication: 24
```

# Recursion in Python

- Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem by breaking it into smaller, simpler subproblems.
- In Python, recursion is especially useful for problems that can be divided into identical smaller tasks, such as mathematical calculations.
- **Working of Recursion**
- A **recursive function** is just like any other Python function except that it calls itself in its body. Let's see basic structure of recursive function:

```
def recursive_function(parameters):  
    if base_case_condition:  
        return base_result  
    else:  
        return recursive_function(modified_parameters)
```

# Recursion in Python

- **Recursive function** contains two key parts:
  - **Base Case:** The stopping condition that prevents infinite recursion.
  - **Recursive Case:** The part of the function where it calls itself with modified parameters.
- **Factorial Calculation**

```
def factorial(n):  
    if n == 0: # Base case  
        return 1  
    else:      # Recursive case  
        return n * factorial(n - 1)  
  
print(factorial(5))
```

120

=== Code

# Recursion in Python

- Fibonacci Sequence

```
def fibonacci(n):  
    if n <= 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
# Print Fibonacci sequence up to n terms  
n_terms = int(input("Enter number of terms: "))  
  
print("Fibonacci sequence:")  
for i in range(n_terms):  
    print(fibonacci(i), end=" ")
```

```
Enter number of terms: 9  
Fibonacci sequence:  
0 1 1 2 3 5 8 13 21
```

# Recursion in Python

- Sum of digits of a number

```
def sum_of_digits(n):  
    if n == 0:  
        return 0  
    return n % 10 + sum_of_digits(n // 10)  
  
print(sum_of_digits(12345))
```

15

=== Code

# Recursion in Python

- Check if a string is palindrome

```
def is_palindrome(s):  
    if len(s) <= 1:  
        return True  
    if s[0] != s[-1]:  
        return False  
    return is_palindrome(s[1:-1])  
  
print(is_palindrome("madam"))  
print(is_palindrome("hello"))
```

Output

True

False

# Recursion in Python

- Print numbers 1 to N

```
def print_numbers(n):  
    if n == 0:  
        return  
    print_numbers(n-1)  
    print(n, end=" ")
```

```
print_numbers(5) |
```

Output

1 2 3 4 5

=== Code Exe

# Python Lambda

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.
- When you need a **short function for temporary use**.

Syntax

`lambda arguments : expression`

```
x = lambda a : a + 10  
print(x(5))
```

Output

15

- Examples:

```
add = lambda x, y: x + y  
print(add(5, 3))
```

Output

8

```
square = lambda n: n * n  
print(square(4))
```

Output

16